

Asynchronous Code

FastAPI Masterclass

I/O-Bound Operations

- Certain operations like querying a database, making an API request, or writing to a file take *time* to complete.
- These are often called **I/O-bound operations** because the program is waiting on input/output from an external resource like the network, database, or file system.
- These operations take an unpredictable amount of time to finish.

Synchronous Operations

- **Synchronous operations** *wait* until the operation is complete before progressing to the next operation.
- Consider the basic execution of a function. Python runs one line after the other.
- An operation that takes some time thus *blocks* the remaining code from executing until it is complete.

Threads I

- Imagine a program needs to handle multiple tasks at the same time. One server example is handling simultaneous requests.
- One solution is to create multiple threads.
- A **thread** is like one “worker” or “unit of work” on a computer.
- A server can create several threads to handle several requests at once.

Threads II

- Threads help, but they do not prevent blocking.
- A thread still *blocks* the next line of code while it waits for I/O. It's still synchronous.
- Threads also have overhead; they require memory and management by the operating system.
- If every thread is waiting on slow I/O, the server can run out of available workers. New requests have to wait.

What is async programming?

- In asynchronous programming, we can start an I/O-bound operation and let Python do *other* work while it completes.
- We currently do **not** have async code. Our Rent a Room API uses synchronous database operations.
- When a route handler calls a method like **session.get**, the thread handling that request blocks until the database responds.

Python's `async` Event Loop

- Python implements an **`async`** event loop that coordinates multiple `async` tasks.
- When one task reaches an I/O operation, it lets another task run while the I/O operation completes.
- The event loop handles the switching back-and-forth automatically. Our code doesn't worry about it.

Async Event Loop and FastAPI

- In FastAPI, we can declare an async route handler with **async def**.
- When the route reaches an awaitable I/O operation, it can pause. We'll discuss this in just a moment!
- While the I/O operation completes, the async event loop can work on another request or I/O operation.
- The server can handle more concurrent I/O requests without dedicating one thread to each request.

Awaitable Functions

- Certain Python packages implement **awaitable** functions/methods.
- We can only use these functions/methods within **async** functions such as route handlers. We prefix the method call with the **await** keyword.
- The awaitable method gives the event loop a place to pause while the operation completes, then resume afterward.

Non-Awaitable Functionality

- We can only use the **await** keyword with awaitable functions/methods.
- Not all packages/libraries provide awaitable functions/methods.
- Python only permits using **await** with awaitable values. The type checker in VSCode will warn us.

def vs async def I

- If a route handler has an awaitable operation, make the function async (**async def**) and **await** the operation.
- If the route handler has blocking code with no support for awaitable operations, make the function sync with **def**.

def vs async def II

- If the route handler has no I/O or blocking code, the function can be either sync (**def**) or async (**async def**).
- For simple immediate code, either approach is fine.
- Most importantly, do **NOT** place blocking, non-awaitable I/O code within an **async** function.